

Angular Note 4.2: Structural Directives

Module: Directives - Structural and Attribute

Topic: Mastering Structural Directives

Duration: 3-4 hours

Introduction

Structural directives are responsible for shaping or reshaping the DOM's structure by adding, removing, or manipulating elements. They are identified by the asterisk (*) prefix and are among the most powerful features in Angular for controlling what appears on the screen.

This note covers the three built-in structural directives (`*ngIf` , `*ngFor` , `*ngSwitch`), their advanced features, and how they work internally.

1. *ngIf - Conditional Rendering

1.1 Basic Usage

The `*ngIf` directive conditionally adds or removes an element from the DOM based on a boolean expression.

Important: `*ngIf` completely removes the element from the DOM when the condition is false, not just hides it with CSS.

Component TypeScript

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
```

```

  })
  export class AppComponent {
    isVisible = true;
    isLoggedIn = false;
    userAge = 25;

    toggle() {
      this.isVisible = !this.isVisible;
    }

    login() {
      this.isLoggedIn = true;
    }

    logout() {
      this.isLoggedIn = false;
    }
  }

```

Component Template

```

<h3>Basic *ngIf</h3>

<button (click)="toggle()">Toggle Content</button>

<div *ngIf="isVisible" style="padding: 15px; background-color: #e8f5e9; margin: 10px 0; border-
radius: 5px;">
  <p>☒ This content is visible because isVisible is true</p>
</div>

<div *ngIf="!isVisible" style="padding: 15px; background-color: #ffebee; margin: 10px 0;
border-radius: 5px;">
  <p>☒ This shows when isVisible is false</p>
</div>

<hr>

<h3>Conditional Expression</h3>

<div *ngIf="userAge >= 18" style="padding: 10px; background-color: #d1ecf1; border-radius:
5px;">
  <p>☒ Adult content: Age is {{ userAge }}</p>
</div>

<div *ngIf="userAge < 18" style="padding: 10px; background-color: #fff3cd; border-radius:

```

```
5px;">
  <p>👤 Minor: Age is {{ userAge }}</p>
</div>
```

Output:

Browser Display:

Initially, a green box appears with "☑ This content is visible because isVisible is true"

A blue box shows "👤 Adult content: Age is 25"

When "Toggle Content" is clicked:

- Green box disappears completely from DOM
- Red box appears with "✗ This shows when isVisible is false"
- Clicking again reverses the change

1.2 *ngIf with Else Block

Angular provides `else` syntax to avoid writing two `*ngIf` directives with opposite conditions.

Component TypeScript

```
export class AppComponent {
  isLoggedIn = false;

  login() {
    this.isLoggedIn = true;
  }

  logout() {
    this.isLoggedIn = false;
  }
}
```

Component Template

```
<h3>*ngIf with Else</h3>

<button (click)="login()" *ngIf="!isLoggedIn">Login</button>
<button (click)="logout()" *ngIf="isLoggedIn">Logout</button>

<div *ngIf="isLoggedIn; else loggedOut"
  style="padding: 15px; background-color: #d4edda; border: 1px solid #c3e6cb; border-radius:
5px; margin: 10px 0;">
  <h4>☑ Welcome back!</h4>
  <p>You have access to all features.</p>
</div>

<ng-template #loggedOut>
  <div style="padding: 15px; background-color: #f8d7da; border: 1px solid #f5c6cb; border-
radius: 5px; margin: 10px 0;">
    <h4>✗ Not logged in</h4>
    <p>Please login to access features.</p>
  </div>
</ng-template>
```

Output:

Browser Display:

Initially (not logged in):

- "Login" button appears
- Red box shows "✗ Not logged in - Please login to access features"

After clicking Login:

- Button changes to "Logout"
- Red box disappears
- Green box appears "☑ Welcome back! - You have access to all features"

1.3 *ngIf with Then and Else

For more complex scenarios, you can use both `then` and `else` templates.

```
<div *ngIf="isLoggedIn; then loggedInTemplate else loggedOutTemplate"></div>

<ng-template #loggedInTemplate>
  <div style="padding: 15px; background-color: #d4edda; border-radius: 5px;">
    <h4>Welcome User!</h4>
  </div>
</ng-template>

<ng-template #loggedOutTemplate>
  <div style="padding: 15px; background-color: #f8d7da; border-radius: 5px;">
    <h4>Guest Mode</h4>
  </div>
</ng-template>
```

1.4 *ngIf with as (Storing Values)

The `as` keyword allows you to store the result of an expression for reuse within the template.

Component TypeScript

```
export class AppComponent {
  currentUser = {
    name: 'John Doe',
    email: 'john@example.com',
    role: 'Admin',
    permissions: ['read', 'write', 'delete']
  };

  user$ = null; // Simulating async data

  loadUser() {
    // Simulating API call
    setTimeout(() => {
      this.user$ = this.currentUser;
    });
  }
}
```

```
    }, 1000);
  }
}
```

Component Template

```
<h3>*ngIf with 'as' keyword</h3>

<button (click)="loadUser()">Load User</button>

<div *ngIf="user$ as user; else loading"
  style="padding: 15px; background-color: #e3f2fd; border-radius: 5px; margin: 10px 0;">
  <h4>User Profile</h4>
  <p><strong>Name:</strong> {{ user.name }}</p>
  <p><strong>Email:</strong> {{ user.email }}</p>
  <p><strong>Role:</strong> {{ user.role }}</p>
  <p><strong>Permissions:</strong> {{ user.permissions.join(', ') }}</p>
</div>

<ng-template #loading>
  <div style="padding: 15px; background-color: #fff3cd; border-radius: 5px; margin: 10px 0;">
    <p>⌚ Loading user data...</p>
  </div>
</ng-template>
```

Output:

Browser Display:

Initially: Yellow box shows "⌚ Loading user data..."

After clicking "Load User": After 1 second, yellow box disappears and blue box appears with:

- User Profile
- Name: John Doe
- Email: john@example.com
- Role: Admin
- Permissions: read, write, delete

1.5 Modern @if Syntax (Angular 17+)

Angular 17 introduced a new control flow syntax using `@if`, `@else if`, and `@else` blocks. This syntax is more concise and doesn't require `ng-template`.

New Feature: The `@if` syntax provides better readability and type safety compared to `*ngIf`.

Component TypeScript

```
export class AppComponent {
  userStatus = 'premium'; // 'guest', 'basic', 'premium', 'admin'
  score = 85;

  changeStatus(status: string) {
    this.userStatus = status;
  }
}
```

Component Template - Basic @if

```
<h3>Modern @if Syntax</h3>

<!-- Simple @if -->
@if (score >= 80) {
  <div style="padding: 15px; background-color: #d4edda; border-radius: 5px; margin: 10px 0;">
    <p>✔ Excellent! Score: {{ score }}</p>
  </div>
}

<!-- @if with @else -->
@if (score >= 60) {
  <div style="padding: 15px; background-color: #d1ecf1; border-radius: 5px;">
    <p>👍 Passing Score: {{ score }}</p>
  </div>
} @else {
  <div style="padding: 15px; background-color: #f8d7da; border-radius: 5px;">
    <p>✖ Needs Improvement: {{ score }}</p>
  </div>
}

<!-- @if with @else if -->
@if (userStatus === 'admin') {
```

```

<div style="padding: 15px; background-color: #e3f2fd; border-radius: 5px; margin: 10px 0;">
  <h4>👑 Admin Dashboard</h4>
  <p>Full system access</p>
</div>
} @else if (userStatus === 'premium') {
  <div style="padding: 15px; background-color: #f3e5f5; border-radius: 5px; margin: 10px 0;">
    <h4>★ Premium User</h4>
    <p>Access to premium features</p>
  </div>
} @else if (userStatus === 'basic') {
  <div style="padding: 15px; background-color: #fff3e0; border-radius: 5px; margin: 10px 0;">
    <h4>📦 Basic User</h4>
    <p>Standard features available</p>
  </div>
} @else {
  <div style="padding: 15px; background-color: #f5f5f5; border-radius: 5px; margin: 10px 0;">
    <h4>👤 Guest</h4>
    <p>Limited access - please sign up</p>
  </div>
}

```

Output:

Browser Display:

Green box: "☑ Excellent! Score: 85"

Purple box: "★ Premium User - Access to premium features"

When userStatus changes, the appropriate box is displayed without needing ng-template.

Comparison: *ngIf vs @if

Traditional *ngIf	Modern @if
<code><div *ngIf="show">Content</div></code>	<code>@if (show) { <div>Content</div> }</code>
<code><div *ngIf="x; else no">Yes</div><ng-template #no>No</ng-template></code>	<code>@if (x) { <div>Yes</div> } @else { <div>No</div> }</code>

Traditional *ngIf	Modern @if
Requires ng-template for multiple conditions	Native @else if support

2. *ngFor - Looping Over Collections

2.1 Basic Usage

The `*ngFor` directive repeats a template for each item in a collection.

Component TypeScript

```
export class AppComponent {
  fruits = ['Apple', 'Banana', 'Orange', 'Mango', 'Grapes'];

  users = [
    { id: 1, name: 'Alice Johnson', age: 28, active: true },
    { id: 2, name: 'Bob Smith', age: 34, active: false },
    { id: 3, name: 'Carol White', age: 25, active: true },
    { id: 4, name: 'David Brown', age: 42, active: true }
  ];
}
```

Component Template

```
<h3>Basic *ngFor</h3>

<h4>Simple List</h4>
<ul>
  <li *ngFor="let fruit of fruits">{{ fruit }}</li>
</ul>

<hr>

<h4>User List</h4>
<div *ngFor="let user of users"
  style="padding: 10px; margin: 10px 0; background-color: #f5f5f5; border-left: 4px solid
#2196f3; border-radius: 5px;">
```

```
<strong>{{ user.name }}</strong> (Age: {{ user.age }})
<span *ngIf="user.active" style="color: green; margin-left: 10px;"><input checked="" type="checkbox"/> Active</span>
<span *ngIf="!user.active" style="color: red; margin-left: 10px;"><input type="checkbox"/> Inactive</span>
</div>
```

Output:

Browser Display:

Simple List:

- Apple
- Banana
- Orange
- Mango
- Grapes

User List:

Four gray boxes with blue left border:

1. Alice Johnson (Age: 28) ☒ Active
2. Bob Smith (Age: 34) ☐ Inactive
3. Carol White (Age: 25) ☒ Active
4. David Brown (Age: 42) ☒ Active

2.2 *ngFor with Index, First, Last, Even, Odd

Angular provides local variables to track position and state within the loop.

Component Template

```
<h3>*ngFor with Local Variables</h3>

<div *ngFor="let fruit of fruits; let i = index; let isFirst = first; let isLast = last; let
isEven = even; let isOdd = odd">
```

```
[style.background-color]="isEven ? '#e3f2fd' : '#fff3e0'"
[style.border-left]="isFirst ? '5px solid green' : isLast ? '5px solid red' : '5px solid #ccc'"
style="padding: 10px; margin: 5px 0; border-radius: 5px;">

<strong>Index {{ i }}:</strong> {{ fruit }}

<span *ngIf="isFirst" style="margin-left: 10px; color: green; font-weight: bold;">[FIRST]</span>
<span *ngIf="isLast" style="margin-left: 10px; color: red; font-weight: bold;">[LAST]</span>
<span style="margin-left: 10px; color: #666;">
  {{ isEven ? '(Even Row)' : '(Odd Row)' }}
</span>
</div>
```

Output:

Browser Display:

1. Index 0: Apple [FIRST] (Even Row) - Light blue background, green left border
2. Index 1: Banana (Odd Row) - Light orange background, gray left border
3. Index 2: Orange (Even Row) - Light blue background, gray left border
4. Index 3: Mango (Odd Row) - Light orange background, gray left border
5. Index 4: Grapes [LAST] (Even Row) - Light blue background, red left border

2.3 *ngFor with trackBy - Performance Optimization

By default, when the collection changes, Angular re-renders all items. The `trackBy` function tells Angular how to track items, preventing unnecessary re-renders.

Performance Tip: Always use `trackBy` when rendering large lists or when the data changes frequently.

Component TypeScript

```
export class AppComponent {
  products = [
    { id: 101, name: 'Laptop', price: 999 },
```

```

    { id: 102, name: 'Mouse', price: 25 },
    { id: 103, name: 'Keyboard', price: 75 }
  ];

  // trackBy function - identifies items by unique id
  trackByProductId(index: number, product: any): number {
    return product.id; // Unique identifier
  }

  // Simulates data refresh (like API call)
  refreshProducts() {
    // Same data but new array instance
    this.products = [
      { id: 101, name: 'Laptop Pro', price: 1099 },
      { id: 102, name: 'Mouse', price: 25 },
      { id: 103, name: 'Keyboard', price: 75 },
      { id: 104, name: 'Monitor', price: 299 }
    ];
  }

  // Alternative: trackBy index (use when no unique identifier)
  trackByIndex(index: number): number {
    return index;
  }
}

```

Component Template

```

<h3>*ngFor with trackBy</h3>

<button (click)="refreshProducts()" style="margin-bottom: 10px;">Refresh Products</button>

<h4>Without trackBy (Re-renders all items) ✗</h4>
<div *ngFor="let product of products"
  style="padding: 10px; margin: 5px 0; background-color: #ffebee; border-radius: 5px;">
  {{ product.name }} - ${{ product.price }}
</div>

<h4>With trackBy (Only re-renders changed items) ✓</h4>
<div *ngFor="let product of products; trackBy: trackByProductId"
  style="padding: 10px; margin: 5px 0; background-color: #e8f5e9; border-radius: 5px;">
  {{ product.name }} - ${{ product.price }}
</div>

```


Output:

Browser Display:

Initially:

Red boxes (without trackBy):

- Laptop - \$999
- Mouse - \$25
- Keyboard - \$75

Green boxes (with trackBy):

- Laptop - \$999
- Mouse - \$25
- Keyboard - \$75

After clicking "Refresh Products":

Both sections update, but with trackBy:

- Items with ID 102 and 103 (Mouse, Keyboard) are NOT re-rendered (same ID)
- Item 101 is updated (Laptop → Laptop Pro, price changed)
- Item 104 (Monitor) is added

Without trackBy, ALL items are destroyed and recreated, even unchanged ones.

2.4 Nested *ngFor

Component TypeScript

```
export class AppComponent {  
  departments = [  
    {  
      name: 'Engineering',  
      teams: ['Frontend', 'Backend', 'DevOps']  
    }  
  ]  
}
```

```
    },  
    {  
      name: 'Marketing',  
      teams: ['Digital', 'Content', 'Social Media']  
    },  
    {  
      name: 'Sales',  
      teams: ['Enterprise', 'SMB', 'Partners']  
    }  
  ];  
}
```

Component Template

```
<h3>Nested *ngFor</h3>  
  
<div *ngFor="let dept of departments; let deptIndex = index"  
  style="margin: 15px 0; padding: 15px; background-color: #f5f5f5; border-radius: 5px;">  
  
  <h4 style="margin-top: 0; color: #2196f3;">  
    {{ deptIndex + 1 }}. {{ dept.name }} Department  
  </h4>  
  
  <ul style="list-style-type: none; padding-left: 0;">  
    <li *ngFor="let team of dept.teams; let teamIndex = index"  
      style="padding: 8px; margin: 5px 0; background-color: white; border-left: 3px solid  
#4caf50; border-radius: 3px;">  
      {{ deptIndex + 1 }}.{{ teamIndex + 1 }} - {{ team }} Team  
    </li>  
  </ul>  
</div>
```

Output:

Browser Display:

1. Engineering Department

- 1.1 - Frontend Team
- 1.2 - Backend Team

- 1.3 - DevOps Team

2. Marketing Department

- 2.1 - Digital Team
- 2.2 - Content Team
- 2.3 - Social Media Team

3. Sales Department

- 3.1 - Enterprise Team
- 3.2 - SMB Team
- 3.3 - Partners Team

2.5 Modern @for Syntax (Angular 17+)

The `@for` syntax provides a more intuitive way to loop over collections with built-in support for empty states and automatic performance optimization.

Advantage: `@for` has built-in `track` requirement and `@empty` block support, eliminating common mistakes.

Component TypeScript

```
export class AppComponent {
  tasks = [
    { id: 1, title: 'Learn Angular', completed: true },
    { id: 2, title: 'Build Project', completed: false },
    { id: 3, title: 'Deploy App', completed: false }
  ];

  emptyList: any[] = [];

  numbers = [1, 2, 3, 4, 5];

  addTask() {
    const newId = this.tasks.length + 1;
    this.tasks.push({
      id: newId,
```

```

    title: `Task ${newId}`,
    completed: false
  });
}

removeTask(id: number) {
  this.tasks = this.tasks.filter(t => t.id !== id);
}
}

```

Component Template - Basic @for

```

<h3>Modern @for Syntax</h3>

<!-- Simple @for with track -->
<h4>Task List</h4>
@for (task of tasks; track task.id) {
  <div style="padding: 10px; margin: 5px 0; background-color: #f5f5f5; border-left: 4px solid
#2196f3; border-radius: 5px;">
    <strong>{{ task.title }}</strong>
    @if (task.completed) {
      <span style="color: green; margin-left: 10px;"><input checked="" type="checkbox"/> Done</span>
    } @else {
      <span style="color: orange; margin-left: 10px;"><input type="checkbox"/> Pending</span>
    }
    <button (click)="removeTask(task.id)" style="margin-left: 10px;">Remove</button>
  </div>
}

<button (click)="addTask()" style="margin: 10px 0;">Add Task</button>

<!-- @for with @empty block -->
<h4>Empty State Example</h4>
@for (item of emptyList; track item.id) {
  <div>{{ item.name }}</div>
} @empty {
  <div style="padding: 20px; background-color: #fff3cd; border-radius: 5px; text-align:
center;">
    <p><input type="checkbox"/> No items to display</p>
    <p>Add some items to get started!</p>
  </div>
}

```

@for with Index and Context Variables

```
<h4>@for with Context Variables</h4>
@for (num of numbers; track num; let idx = $index; let isFirst = $first; let isLast = $last;
let isEven = $even) {
  <div
    [style.background-color]="isEven ? '#e3f2fd' : '#fff3e0'"
    [style.border-left]="isFirst ? '5px solid green' : isLast ? '5px solid red' : '5px solid
#ccc'"
    style="padding: 10px; margin: 5px 0; border-radius: 5px;">

    <strong>Index {{ idx }}:</strong> Number {{ num }}

    @if (isFirst) {
      <span style="margin-left: 10px; color: green; font-weight: bold;">[FIRST]</span>
    }
    @if (isLast) {
      <span style="margin-left: 10px; color: red; font-weight: bold;">[LAST]</span>
    }
    <span style="margin-left: 10px; color: #666;">
      {{ isEven ? '(Even)' : '(Odd)' }}
    </span>
  </div>
}
```

Output:

Browser Display:

Task List:

1. Learn Angular ☒ Done [Remove]
2. Build Project ⌚ Pending [Remove]
3. Deploy App ⌚ Pending [Remove]

[Add Task] button

Empty State:

Yellow box: "🔍 No items to display - Add some items to get started!"

Context Variables:

1. Index 0: Number 1 [FIRST] (Even) - Blue, green border
2. Index 1: Number 2 (Odd) - Orange, gray border
3. Index 2: Number 3 (Even) - Blue, gray border
4. Index 3: Number 4 (Odd) - Orange, gray border
5. Index 4: Number 5 [LAST] (Even) - Blue, red border

Comparison: *ngFor vs @for

Traditional *ngFor	Modern @for
<code><div *ngFor="let item of items"></code>	<code>@for (item of items; track item.id) { <div> }</code>
trackBy is optional (but recommended)	track is REQUIRED
<code>let i = index</code>	<code>let i = \$index</code>
<code>let f = first</code>	<code>let f = \$first</code>
<code>let e = even</code>	<code>let e = \$even</code>
No built-in empty state	<code>@empty { } block</code>
<code>let c = count</code>	<code>let c = \$count</code>

3. *ngSwitch - Switch-Case Rendering

3.1 Basic Usage

The `*ngSwitch` directive displays one element from multiple possible elements based on a switch condition.

Component TypeScript

```
export class AppComponent {
  selectedView = 'home';
  userRole = 'user';
  subscriptionTier = 'premium';

  changeView(view: string) {
    this.selectedView = view;
  }

  changeRole(role: string) {
    this.userRole = role;
  }

  changeTier(tier: string) {
    this.subscriptionTier = tier;
  }
}
```

Component Template

```
<h3>*ngSwitch Example</h3>

<div style="margin-bottom: 15px;">
  <button (click)="changeView('home')">Home</button>
  <button (click)="changeView('profile')">Profile</button>
  <button (click)="changeView('settings')">Settings</button>
  <button (click)="changeView('about')">About</button>
</div>

<div [ngSwitch]="selectedView">
  <div *ngSwitchCase="'home'"
    style="padding: 20px; background-color: #e3f2fd; border-radius: 5px;">
    <h4><img alt="Home icon" data-bbox="175 700 195 715"/> Home Page</h4>
    <p>Welcome to the home page!</p>
  </div>

  <div *ngSwitchCase="'profile'"
    style="padding: 20px; background-color: #f3e5f5; border-radius: 5px;">
    <h4><img alt="User icon" data-bbox="175 795 195 810"/> User Profile</h4>
    <p>Manage your profile settings here.</p>
  </div>
</div>
```

```
<div *ngSwitchCase="'settings'"
  style="padding: 20px; background-color: #fff3e0; border-radius: 5px;">
  <h4>⚙️ Settings</h4>
  <p>Configure application preferences.</p>
</div>

<div *ngSwitchDefault
  style="padding: 20px; background-color: #f5f5f5; border-radius: 5px;">
  <h4>❗ Default View</h4>
  <p>No matching page found. Showing default content.</p>
</div>
</div>
```

Output:

Browser Display:

Four buttons appear: Home, Profile, Settings, About

Initially (Home selected):

Light blue box: "🏠 Home Page - Welcome to the home page!"

After clicking "Profile":

Purple box: "👤 User Profile - Manage your profile settings here."

After clicking "Settings":

Orange box: "⚙️ Settings - Configure application preferences."

After clicking "About":

Gray box: "❗ Default View - No matching page found. Showing default content."

3.2 Complex *ngSwitch Example

Component Template

```
<h3>Subscription Dashboard</h3>

<div style="margin-bottom: 15px;">
  <button (click)="changeTier('free')">Free</button>
  <button (click)="changeTier('basic')">Basic</button>
  <button (click)="changeTier('premium')">Premium</button>
  <button (click)="changeTier('enterprise')">Enterprise</button>
</div>

<div [ngSwitch]="subscriptionTier">
  <div *ngSwitchCase="'free'"
    style="padding: 20px; background-color: #e0e0e0; border: 2px solid #9e9e9e; border-
radius: 8px;">
    <h4>📁 Free Plan</h4>
    <ul>
      <li>5 Projects</li>
      <li>Basic Support</li>
      <li>Community Access</li>
    </ul>
    <p><strong>Price: $0/month</strong></p>
  </div>

  <div *ngSwitchCase="'basic'"
    style="padding: 20px; background-color: #e8f5e9; border: 2px solid #4caf50; border-
radius: 8px;">
    <h4>📁 Basic Plan</h4>
    <ul>
      <li>25 Projects</li>
      <li>Email Support</li>
      <li>Premium Templates</li>
    </ul>
    <p><strong>Price: $9/month</strong></p>
  </div>

  <div *ngSwitchCase="'premium'"
    style="padding: 20px; background-color: #e3f2fd; border: 2px solid #2196f3; border-
radius: 8px;">
    <h4>★ Premium Plan</h4>
    <ul>
      <li>Unlimited Projects</li>
      <li>Priority Support</li>
    </ul>
  </div>
</div>
```

```

    <li>Advanced Analytics</li>
    <li>Custom Domain</li>
  </ul>
  <p><strong>Price: $29/month</strong></p>
</div>

<div *ngSwitchCase="'enterprise'"
  style="padding: 20px; background-color: #f3e5f5; border: 2px solid #9c27b0; border-
radius: 8px;">
  <h4>🏢 Enterprise Plan</h4>
  <ul>
    <li>Unlimited Everything</li>
    <li>24/7 Dedicated Support</li>
    <li>Custom Solutions</li>
    <li>SLA Guarantee</li>
    <li>Training & Onboarding</li>
  </ul>
  <p><strong>Price: Contact Sales</strong></p>
</div>
</div>

```

Output:

Browser Display:

Four subscription tier buttons appear.

Premium Plan (Initially):

Blue box with border showing:

- ★ Premium Plan
- Unlimited Projects
- Priority Support
- Advanced Analytics
- Custom Domain
- Price: \$29/month

Clicking different tier buttons changes the displayed plan with appropriate styling.

3.3 Modern @switch Syntax (Angular 17+)

The `@switch` syntax simplifies multi-branch conditional rendering with cleaner syntax and better performance.

Component TypeScript

```
export class AppComponent {
  paymentMethod = 'credit'; // 'credit', 'debit', 'paypal', 'crypto'
  orderStatus = 'processing'; // 'pending', 'processing', 'shipped', 'delivered'

  changePayment(method: string) {
    this.paymentMethod = method;
  }

  changeOrderStatus(status: string) {
    this.orderStatus = status;
  }
}
```

Component Template

```
<h3>Modern @switch Syntax</h3>

<div style="margin-bottom: 15px;">
  <button (click)="changePayment('credit')">Credit Card</button>
  <button (click)="changePayment('debit')">Debit Card</button>
  <button (click)="changePayment('paypal')">PayPal</button>
  <button (click)="changePayment('crypto')">Cryptocurrency</button>
</div>

@switch (paymentMethod) {
  @case ('credit') {
    <div style="padding: 20px; background-color: #e3f2fd; border: 2px solid #2196f3; border-radius: 8px;">
      <h4>🏠 Credit Card</h4>
      <p>Accepted: Visa, MasterCard, Amex</p>
      <p>Processing fee: 2.9% + $0.30</p>
    </div>
  }
  @case ('debit') {
    <div style="padding: 20px; background-color: #e8f5e9; border: 2px solid #4caf50; border-
```

```

radius: 8px;">
  <h4>🏠 Debit Card</h4>
  <p>Direct bank payment</p>
  <p>Processing fee: 1.5% + $0.25</p>
</div>
}
@case ('paypal') {
  <div style="padding: 20px; background-color: #fff3e0; border: 2px solid #ff9800; border-
radius: 8px;">
    <h4>💳 PayPal</h4>
    <p>Secure online payment</p>
    <p>Processing fee: 3.5% + $0.35</p>
  </div>
}
@case ('crypto') {
  <div style="padding: 20px; background-color: #f3e5f5; border: 2px solid #9c27b0; border-
radius: 8px;">
    <h4>₿ Cryptocurrency</h4>
    <p>Bitcoin, Ethereum, USDC</p>
    <p>Network fee: varies</p>
  </div>
}
@default {
  <div style="padding: 20px; background-color: #f5f5f5; border: 2px solid #9e9e9e; border-
radius: 8px;">
    <h4>? Unknown Payment Method</h4>
    <p>Please select a valid payment option</p>
  </div>
}
}

<hr>

<h4>Order Status Tracker</h4>

<div style="margin-bottom: 15px;">
  <button (click)="changeOrderStatus('pending')">Pending</button>
  <button (click)="changeOrderStatus('processing')">Processing</button>
  <button (click)="changeOrderStatus('shipped')">Shipped</button>
  <button (click)="changeOrderStatus('delivered')">Delivered</button>
</div>

@switch (orderStatus) {
  @case ('pending') {
    <div style="padding: 15px; background-color: #fff3cd; border-left: 5px solid #ffc107;
border-radius: 5px;">
      <h4>🕒 Order Pending</h4>

```

```

    <p>Your order is waiting for confirmation</p>
  </div>
}
@case ('processing') {
  <div style="padding: 15px; background-color: #d1ecf1; border-left: 5px solid #17a2b8;
border-radius: 5px;">
    <h4>📦 Processing Order</h4>
    <p>We're preparing your items for shipment</p>
  </div>
}
@case ('shipped') {
  <div style="padding: 15px; background-color: #e3f2fd; border-left: 5px solid #2196f3;
border-radius: 5px;">
    <h4>📦 Order Shipped</h4>
    <p>Your package is on the way!</p>
  </div>
}
@case ('delivered') {
  <div style="padding: 15px; background-color: #d4edda; border-left: 5px solid #28a745;
border-radius: 5px;">
    <h4>✅ Order Delivered</h4>
    <p>Your order has been delivered successfully</p>
  </div>
}
@default {
  <div style="padding: 15px; background-color: #f8d7da; border-left: 5px solid #dc3545;
border-radius: 5px;">
    <h4>❌ Unknown Status</h4>
    <p>Unable to determine order status</p>
  </div>
}
}

```

Output:

Browser Display:

Four payment method buttons appear at the top.

Initially (Credit Card):

Blue box: "📦 Credit Card - Accepted: Visa, MasterCard, Amex - Processing fee: 2.9% + \$0.30"

Order Status (Processing):

Light blue box: "📦 Processing Order - We're preparing your items for shipment"

Clicking different buttons changes the displayed content with appropriate styling and information.

Comparison: *ngSwitch vs @switch

Traditional *ngSwitch	Modern @switch
<code><div [ngSwitch]="value"><div *ngSwitchCase="'a'"></code>	<code>@switch (value) { @case ('a') { <div> } }</code>
<code>*ngSwitchCase</code> on each element	<code>@case</code> blocks
<code>*ngSwitchDefault</code>	<code>@default</code>
Requires container element with <code>[ngSwitch]</code>	Standalone <code>@switch</code> block
More verbose	More concise and readable

4. How Structural Directives Work: * Syntax and ng-template

4.1 Understanding the Asterisk (*)

The asterisk (`*`) is syntactic sugar that Angular transforms into a more verbose form using `<ng-template>` .

What You Write (Shorthand):

```
<div *ngIf="condition">Content</div>
```

What Angular Transforms It To:

```
<ng-template [ngIf]="condition">
  <div>Content</div>
</ng-template>
```

4.2 ng-template Explained

`<ng-template>` is an Angular element that holds content but doesn't render it by default. Structural directives use it to determine what to show or hide.

Manual ng-template Usage

```
<h3>Manual ng-template</h3>

<!-- This content is NOT rendered -->
<ng-template>
  <p>This is inside ng-template and won't show</p>
</ng-template>

<!-- Using template reference with ngIf -->
<div *ngIf="isVisible; else hiddenContent">
  <p>Visible content</p>
</div>

<ng-template #hiddenContent>
  <p>This shows when isVisible is false</p>
</ng-template>
```

4.3 Desugaring Examples

Shorthand (With *)	Desugared (With ng-template)
<code><div *ngIf="show">Hi</div></code>	<code><ng-template [ngIf]="show"><div>Hi</div></ng-template></code>
<code><li *ngFor="let i of items">{{i}}</code>	<code><ng-template ngFor let-i [ngForOf]="items">{{i}}</ng-template></code>

Shorthand (With *)	Desugared (With ng-template)
<code><div *ngSwitchCase="'a'">A</div></code>	<code><ng-template [ngSwitchCase]="'a'"><div>A</div></ng-template></code>

4.4 Why Only One Structural Directive Per Element

Since each structural directive wraps the element in `<ng-template>`, having multiple would create ambiguous nesting:

```
<!-- ✗ INVALID -->
<div *ngIf="condition" *ngFor="let item of items"></div>

<!-- Would desugar to ambiguous structure -->
<ng-template [ngIf]="condition">
  <ng-template ngFor let-item [ngForOf]="items">
    <div></div>
  </ng-template>
</ng-template>

<!-- ✓ CORRECT - Use container element -->
<div *ngIf="condition">
  <div *ngFor="let item of items"></div>
</div>

<!-- ✓ ALTERNATIVE - Use ng-container (no extra DOM element) -->
<ng-container *ngIf="condition">
  <div *ngFor="let item of items"></div>
</ng-container>
```

4.5 Using ng-container

`<ng-container>` is a logical grouping element that doesn't create an actual DOM element.

Component TypeScript

```
export class AppComponent {  
  showList = true;  
  items = ['Item 1', 'Item 2', 'Item 3'];  
}
```

Component Template

```
<h3>ng-container Example</h3>  
  
<button (click)="showList = !showList">Toggle List</button>  
  
<!-- Without ng-container (adds extra div) -->  
<div style="background-color: #ffebee; padding: 10px; margin: 10px 0;">  
  <h4>With Extra Div (Not Ideal)</h4>  
  <div *ngIf="showList">  
    <ul>  
      <li *ngFor="let item of items">{{ item }}</li>  
    </ul>  
  </div>  
</div>  
  
<!-- With ng-container (no extra DOM element) -->  
<div style="background-color: #e8f5e9; padding: 10px; margin: 10px 0;">  
  <h4>With ng-container (Clean DOM)</h4>  
  <ng-container *ngIf="showList">  
    <ul>  
      <li *ngFor="let item of items">{{ item }}</li>  
    </ul>  
  </ng-container>  
</div>
```

Output:

Browser Display:

Both sections show the list initially. However, inspecting the DOM reveals:

Without ng-container:

```
<div>
  <h4>With Extra Div</h4>
  <div> <!-- Extra wrapper div -->
    <ul>...</ul>
  </div>
</div>
```

With ng-container:

```
<div>
  <h4>With ng-container</h4>
  <ul>...</ul> <!-- No extra wrapper -->
</div>
```

5. Best Practices

5.1 Performance Tips

- Always use `trackBy` with `*ngFor` for large lists
- Prefer `*ngIf` over `[hidden]` for elements that change infrequently
- Use `ng-container` to avoid extra DOM elements
- Keep template expressions simple and move complex logic to components

5.2 When to Use Each Directive

Directive	Use Case
<code>*ngIf</code>	Show/hide elements based on single condition
<code>*ngFor</code>	Display lists or collections
<code>*ngSwitch</code>	Show one of many possible elements (3+ options)

5.3 Common Mistakes to Avoid

1. Multiple structural directives on one element

```
<!-- ✗ Wrong -->
<div *ngIf="show" *ngFor="let item of items"></div>

<!-- ✓ Correct -->
<ng-container *ngIf="show">
  <div *ngFor="let item of items"></div>
</ng-container>
```

2. Not using trackBy with *ngFor

```
<!-- ✗ Performance issue -->
<div *ngFor="let item of items"></div>

<!-- ✓ Optimized -->
<div *ngFor="let item of items; trackBy: trackById"></div>
```

3. Complex expressions in templates

```
<!-- ✗ Hard to read -->
<div *ngIf="user && user.age > 18 && user.verified && !user.suspended"></div>

<!-- ✓ Use component method -->
<div *ngIf="isEligibleUser()"></div>
```

Summary

Structural directives are powerful tools for controlling DOM structure:

1. ***ngIf / @if**: Conditional rendering with else blocks and as keyword for storing values. Modern @if syntax supports @else if and @else with cleaner syntax
2. ***ngFor / @for**: Loop over collections with index, first, last, even, odd variables. Always use trackBy for *ngFor. Modern @for requires track and supports @empty block

3. ***ngSwitch / @switch**: Choose one element from multiple options based on expression value. Modern @switch uses @case and @default blocks
4. **Modern Syntax (Angular 17+)**: The @if, @for, and @switch syntax provides better readability, type safety, and built-in optimizations
5. **Internal Mechanism**: The asterisk (*) is shorthand for `<ng-template>`. Angular transforms structural directives into template-based rendering
6. **ng-container**: Use for structural directives without adding extra DOM elements

When to Use Traditional vs Modern Syntax

Scenario	Recommendation
New projects (Angular 17+)	Use modern @syntax for better readability
Existing projects	Both syntaxes work - migrate gradually
Complex conditionals	@if with @else if is more readable than nested *ngIf
Lists with empty states	@for with @empty eliminates need for separate check
Multiple options (3+)	@switch is cleaner than multiple *ngIf

Mastering structural directives is essential for building dynamic, performant Angular applications.

End of Angular Note 4.2: Structural Directives